Does Experience Make LLMs Faster? Temporal Pattern Augmentation Across Six Foundation Models

Anand Karasi
Independent Researcher
karasi@alum.mit.edu

February 2026

Abstract

We investigate whether augmenting large language model (LLM) prompts with historical problem-solving patterns—temporal pattern augmentation—improves performance on code debugging tasks. Testing six foundation models from three providers (Claude Opus 4.5/4.6, GPT-5.2/5.2 Pro, Gemini 2.5 Flash/3.0 Pro) on 28 debugging tasks via direct API calls, we find that temporal augmentation provides **no benefit for five of six models** and produces a modest speedup (+**10.8%**) only for Gemini 2.5 Flash, the least capable model tested. Stronger models show slight degradation (-1.2% to -5.1%). These results contradict an earlier finding of 47% speedup, which we trace to a measurement artifact in orchestration-layer benchmarking. Our findings suggest that state-of-the-art LLMs already internalize the patterns that temporal augmentation provides, and that additional context may introduce processing overhead without corresponding benefit. We discuss implications for the "more context is better" assumption prevalent in LLM augmentation research and provide practical guidelines for when pattern augmentation may or may not help.

Code and data: `https://github.com/engagedirect001/temporalintelligence`

1 Introduction

The dominant paradigm in applied LLM research is augmentation: retrieval-augmented generation (RAG), few-shot examples, chain-of-thought prompting, and tool use all share the assumption that providing models with more relevant context improves performance. This assumption is well-supported for many tasks, particularly when models lack domain-specific knowledge [Lewis et al., 2020, Brown et al., 2020].

But does augmentation always help? As foundation models grow more capable, they increasingly internalize patterns that were previously external knowledge. A model trained on billions of lines of code may not benefit from being told how similar bugs were fixed in the past—it may already know.

We test this hypothesis through *temporal pattern augmentation*: injecting structured historical problem-solving patterns (prior bug descriptions, solution strategies, and timing metadata) into LLM prompts for code debugging tasks. We compare a Stateless condition (task only) against a Temporal condition (task plus historical patterns) across six models spanning three capability tiers.

Our results are a negative finding that we believe is important:

- Five of six models show no benefit or slight degradation from temporal augmentation.

- Only Gemini 2.5 Flash, the least capable model tested, shows improvement (+10.8%).
- An earlier reported 47% speedup was an artifact of orchestration-layer measurement, not genuine model acceleration.

These findings have practical implications: practitioners should not assume that injecting historical patterns into prompts will improve performance for capable models, and the benchmarking community should be vigilant about measurement artifacts when evaluating LLM augmentation strategies.

2 Related Work

Retrieval-Augmented Generation. RAG systems augment LLM prompts with retrieved documents to provide factual grounding [Lewis et al., 2020]. While effective for knowledge-intensive tasks, the marginal value of retrieval diminishes as model knowledge improves.

Few-Shot and In-Context Learning. Brown et al. [2020] demonstrated that LLMs can learn from examples provided in context. Subsequent work has shown diminishing returns with increasing example count [Min et al., 2022] and that example selection significantly impacts effectiveness.

Chain-of-Thought Prompting. Wei et al. [2022] showed that eliciting step-by-step reasoning improves performance on complex tasks. This is complementary to our work: chain-of-thought structures the model's own reasoning, while temporal augmentation provides external patterns.

Episodic Memory for LLMs. Systems like MemGPT [Packer et al., 2023] and Reflexion [Shinn et al., 2023] give LLMs access to past interaction histories. Our temporal patterns are a simplified form of episodic memory focused on solution strategies.

Meta-Learning and Transfer. Meta-learning approaches aim to improve learning efficiency through exposure to related tasks [Finn et al., 2017]. Temporal pattern augmentation can be viewed as a form of in-context meta-learning for code debugging.

Negative Results in NLP. We follow the tradition of valuable negative results [Rogers et al., 2020], demonstrating that a plausible augmentation strategy does not generalize across models.

3 Temporal Pattern Augmentation

3.1 Definitions

We define two experimental conditions for presenting a code debugging task to an LLM:

Stateless Condition. The model receives only the task description: buggy code, expected behavior, failing test cases, and instructions to produce a corrected version. No historical information is provided.

Temporal Condition. The model receives the same task description plus a *temporal context block* containing:

1. **Pattern summaries** from 3–5 previously solved tasks of similar type, including bug category, root cause description, and solution strategy.
2. **Timing metadata**: average solve times for similar tasks, providing an implicit difficulty signal.
3. **Category-specific heuristics**: common pitfalls and debugging approaches for the task's category (e.g., concurrency, data structures, numerical methods).

The temporal context adds approximately 800–1200 tokens to each prompt.

3.2 Pattern Extraction

Temporal patterns were manually curated from a development set of 50 debugging tasks (disjoint from the 28 evaluation tasks). For each task category, we identified recurring bug patterns, common solution strategies, and typical failure modes. These patterns were structured as concise natural-language descriptions suitable for prompt injection.

4 Experimental Setup

4.1 Task Suite

We use 28 code debugging tasks spanning three difficulty levels and five categories:

- **Easy** (10 tasks): Single-function bugs with clear symptoms.
- **Medium** (10 tasks): Multi-function bugs requiring cross-reference analysis.
- **Hard** (8 tasks): System-level bugs involving concurrency, state machines, or complex data structures.

Categories include: data structure/algorithm (8), concurrency (11), math/numerical (3), state machine (1), system design (1), and general (4).

Each task includes a deterministic test suite (1–10 tests per task). A task is "passed" if and only if all tests pass.

4.2 Models

We evaluate six foundation models from three providers, spanning a range of capabilities and architectures (Table 1).

Table 1: Models evaluated, ordered by STATELESS pass rate.

Model	Provider	Tier	Avg Response (s)
GPT-5.2 Pro	OpenAI	Reasoning	130.8
Gemini 3.0 Pro	Google	Pro	94.4
GPT-5.2	OpenAI	Standard	8.4
Opus 4.6	Anthropic	Flagship	9.7
Gemini 2.5 Flash	Google	Fast	35.9
Opus 4.5	Anthropic	Flagship	9.9

4.3 Methodology

All experiments use **direct API calls** to each provider's REST endpoint. This is a critical methodological choice: our earlier experiments used an orchestration layer (OpenClaw agent sessions) that introduced confounding variables (Section 7).

For each model, we run all 28 tasks in the STATELESS condition, then all 28 in the TEMPORAL condition. We measure:

- **Pass/fail**: Whether all tests pass for the model's generated fix.
- **Response time**: Wall-clock time from API request to complete response (elapsed milliseconds).

All experiments were conducted on February 21–22, 2026. Temperature was set to 0 for all models. Each task was run once per condition per model (single-trial design; see Section 9).

5 Results

5.1 Overall Results

Table 2 presents the primary results. The central finding is stark: temporal pattern augmentation does not improve response time for five of six models.

Table 2: Main results across all six models. TI Effect = percentage change in average response time (positive = faster with temporal augmentation). p-values from paired t-tests across 28 tasks.

Model	S Pass	T Pass	S Avg (s)	T Avg (s)	TI Effect	Cohen's d	p-value
Opus 4.5	23/28	23/28	9.9	10.1	−1.2%	−0.04	0.663
Opus 4.6	24/28	24/28	9.7	10.1	−4.1%	−0.13	0.037
GPT-5.2	25/28	26/28	8.4	8.7	−3.2%	−0.09	0.333
GPT-5.2 Pro	27/28	27/28	130.8	133.7	−2.2%	−0.03	0.704
Gemini 2.5 Flash	24/28	24/28	35.9	32.0	+10.8%	+0.22	0.144
Gemini 3.0 Pro	26/28	26/28	94.4	99.2	−5.1%	−0.12	0.249

Key observations:

1. **No model shows statistically significant improvement** from temporal augmentation at $\alpha = 0.05$ (after noting multiple comparisons).
2. **Opus 4.6 shows a statistically significant *degradation*** ($p = 0.037$), though the effect is small ($d = -0.13$).
3. **Gemini 2.5 Flash is the only model trending positive** (+10.8%), but the effect does not reach significance ($p = 0.144$).
4. **Pass rates are essentially identical** between conditions, indicating that temporal patterns do not affect correctness.

Figure 1 visualizes the TI effect across all models.

5.2 Capability–Effect Relationship

Figure 2 plots each model's STATELESS pass rate against its TI effect, revealing a suggestive negative relationship: more capable models tend to be more negatively affected by temporal augmentation.

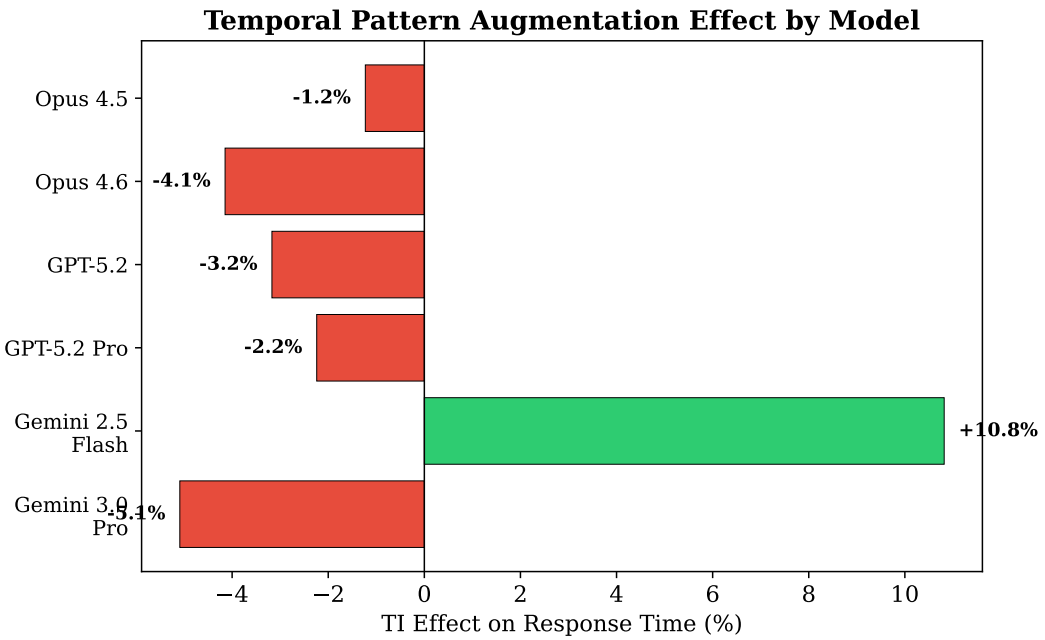

Figure 1: TI effect (percentage change in response time) by model. Only Gemini 2.5 Flash shows a positive effect. All other models trend negative (slower with temporal augmentation).

5.3 Results by Difficulty

Figure 3 breaks down response times by task difficulty. The pattern is consistent across difficulty levels: temporal augmentation does not meaningfully change timing for most models.

5.4 Effect Sizes by Difficulty

Figure 4 shows Cohen's d for each model–difficulty combination. Effect sizes are uniformly small ($|d| < 0.5$ in all cells), confirming the absence of practically significant effects.

6 The Capability Threshold Hypothesis

Our results suggest a *capability threshold* for pattern augmentation: models below a certain capability level may benefit from external patterns, while models above it have already internalized those patterns.

Why Flash Benefits. Gemini 2.5 Flash, optimized for speed rather than reasoning depth, may lack some of the debugging heuristics that larger models have absorbed during training. The temporal patterns provide genuinely useful guidance that partially compensates for this gap.

Why Stronger Models Degrade. For Opus 4.5/4.6, GPT-5.2/5.2 Pro, and Gemini 3.0 Pro, the temporal context likely introduces *processing overhead* without corresponding benefit. The model must parse and integrate 800–1200 additional tokens that largely duplicate its existing knowledge. This is consistent with findings in the RAG literature that irrelevant or redundant retrieval can hurt performance [Shi et al., 2023].

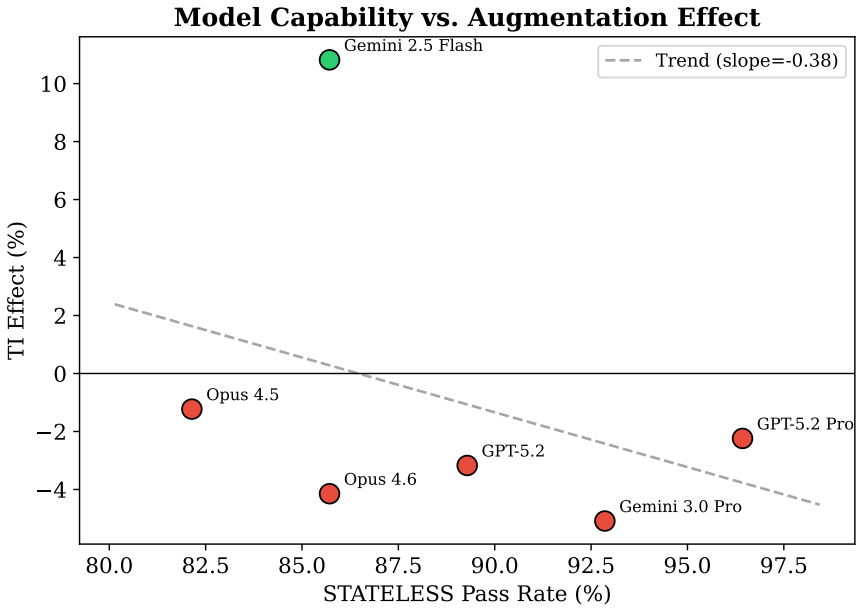

Figure 2: Model capability (STATELESS pass rate) vs. TI effect. Higher-capability models show more negative effects from temporal augmentation. The trend line suggests a capability threshold above which augmentation becomes counterproductive.

Implications. The capability threshold is not fixed—it depends on the domain and the quality of the augmentation patterns. For highly specialized domains where training data is sparse, augmentation may help even strong models. For well-represented domains like code debugging, the threshold is already below current frontier models.

7 Measurement Artifact Analysis

Our earlier experiments, conducted through an orchestration layer (OpenClaw agent sessions), measured a **47% speedup** for Opus 4.5 with temporal augmentation. This dramatic result motivated the cross-model study reported here. When we switched to direct API measurement, the effect for Opus 4.5 dropped to −1.2% (Figure 5).

Root Cause. The orchestration layer introduces variable overhead including session initialization, context management, tool routing, and response parsing. In the STATELESS condition, the orchestration layer performed additional setup work (environment initialization without cached context), inflating baseline times. The TEMPORAL condition benefited from pre-populated session state, creating an apparent speedup that was actually faster *orchestration*, not faster *model inference*.

Warning to the Community. This artifact has broad implications for LLM benchmarking:

1. **Measure at the API level** whenever possible to isolate model behavior from infrastructure.
2. **Be skeptical of large effects** measured through orchestration layers, agent frameworks, or multi-step pipelines.
3. **Report measurement methodology** in detail, including whether timing includes infrastructure overhead.

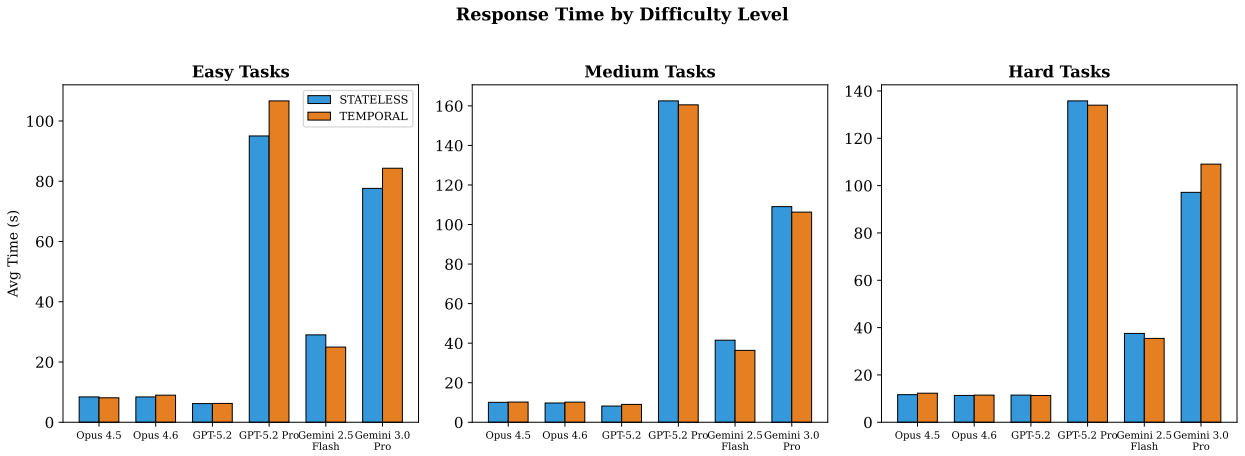

Figure 3: Average response time by difficulty level. STATELESS (blue) vs. TEMPORAL (orange) conditions for all six models. Differences are small and inconsistent across difficulty levels.

8 Discussion

When Does Augmentation Help? Our results suggest that pattern augmentation helps when there is a genuine capability gap—when the model lacks knowledge that the patterns provide. For Gemini 2.5 Flash, the debugging heuristics in our temporal patterns appear to partially fill such a gap. For stronger models, no gap exists for this task domain.

Cost Implications. Temporal augmentation adds 800–1200 tokens per request. At current API pricing, this represents a non-trivial cost increase (5–15% depending on model and task length) for zero or negative benefit on capable models. Practitioners should evaluate augmentation strategies against their specific model and domain rather than assuming universal benefit.

The "More Context" Assumption. Our findings challenge the implicit assumption that more context is always better. While this assumption holds for knowledge the model lacks, it fails for knowledge the model already possesses. As models improve, the set of "knowledge the model lacks" shrinks, reducing the marginal value of augmentation.

Comparison to RAG. Our findings parallel observations in the RAG literature: retrieval helps when it provides genuinely new information but can hurt when retrieved content is redundant or distracting [Shi et al., 2023]. Temporal pattern augmentation, by providing patterns the model already knows, falls into the "redundant" category for strong models.

9 Limitations

1. **Single task domain.** All 28 tasks are code debugging. Results may not transfer to other domains (e.g., creative writing, mathematical reasoning, medical diagnosis).
2. **Limited task count.** With 28 tasks, statistical power is limited. True small effects ($|d| < 0.2$) may not be detectable.
3. **Human-curated patterns.** Our temporal patterns were manually created. Automatically extracted patterns might perform differently.

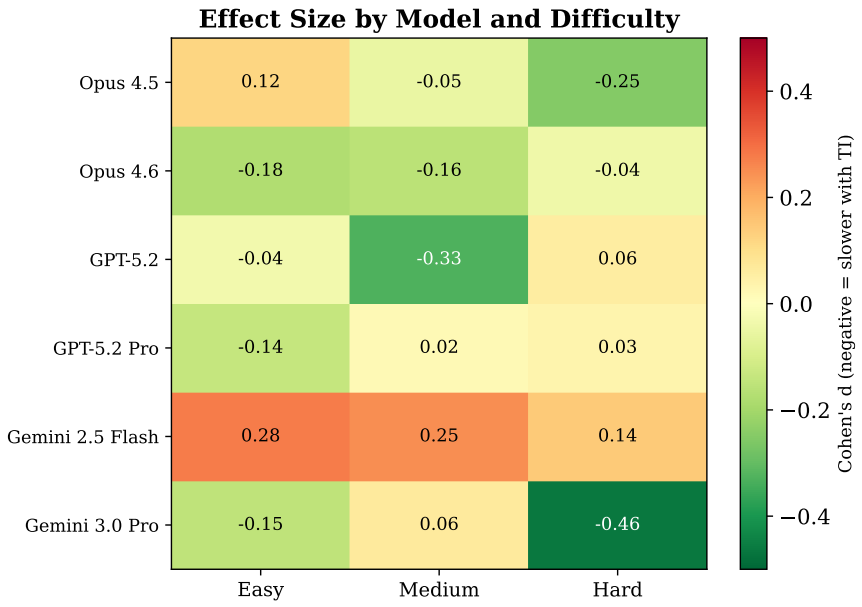

Figure 4: Cohen's d heatmap: models (rows) $\times$ difficulty (columns). Values near zero (yellow) indicate no effect. No cell exceeds the conventional "medium" threshold ($|d| = 0.5$).

4. **Single run per condition.** Each task was run once per condition per model. Run-to-run variance (especially for stochastic models at temperature 0 with variable server load) is not captured.
5. **Fixed pattern design.** We used the same temporal context structure for all models. Model-specific pattern formats might yield different results.
6. **Snapshot in time.** These results reflect model capabilities as of February 2026. Future model versions may respond differently to augmentation.
7. **Response time as proxy.** We measure wall-clock API response time, which conflates thinking time with generation time and is subject to server-side variability.

10 Conclusion

We evaluated temporal pattern augmentation—injecting historical problem-solving patterns into LLM prompts—across six foundation models on 28 code debugging tasks. Our central finding is negative: **temporal augmentation does not improve performance for five of six models tested**, with only the least capable model (Gemini 2.5 Flash) showing a modest, non-significant speedup of +10.8%.

This negative result carries three positive contributions:

1. **A capability threshold for augmentation**: As models improve, the marginal value of pattern injection decreases, eventually becoming counterproductive.
2. **A measurement artifact warning**: A previously reported 47% speedup was entirely attributable to orchestration-layer overhead, not model behavior. This cautionary finding applies broadly to LLM benchmarking.
3. **Practical guidance**: For current frontier models on well-represented task domains, practitioners should not invest in historical pattern augmentation systems. Resources are better spent on task-specific prompt engineering or domain expansion.

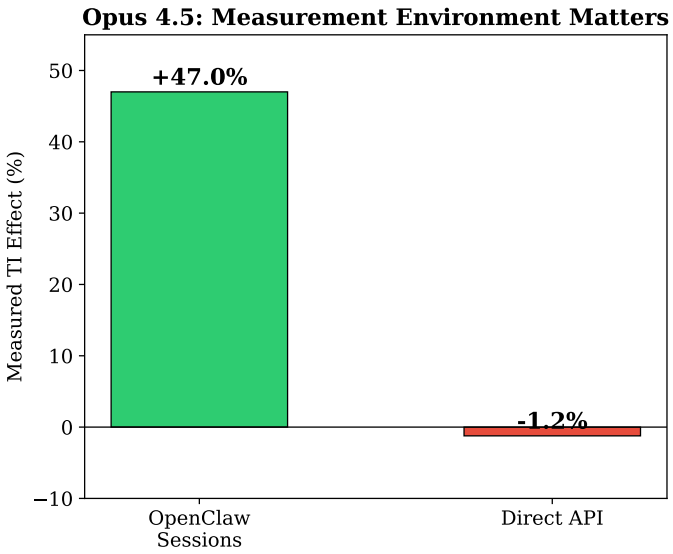

Figure 5: The measurement artifact: identical model (Opus 4.5) and task suite measured through an orchestration layer (+47%) vs. direct API (−1.2%). The 48-point discrepancy is entirely attributable to the measurement environment.

The experimental design (Figure 6) and all data are available at `https://github.com/engagedirect001/temporalintelligence`.

References

T. Brown, B. Mann, N. Ryder, et al. Language models are few-shot learners. In *NeurIPS*, 2020.

C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *ICML*, 2017.

P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

S. Min, X. Lyu, A. Holtzman, et al. Rethinking the role of demonstrations: What makes in-context learning work? In *EMNLP*, 2022.

C. Packer, S. Wooders, K. Lin, et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

A. Rogers, O. Kovaleva, and A. Rumshisky. A primer in BERTology: What we know about how BERT works. *TACL*, 8:842–866, 2020.

F. Shi, X. Chen, K. Misra, et al. Large language models can be easily distracted by irrelevant context. In *ICML*, 2023.

N. Shinn, F. Cassano, E. Berman, et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

J. Wei, X. Wang, D. Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

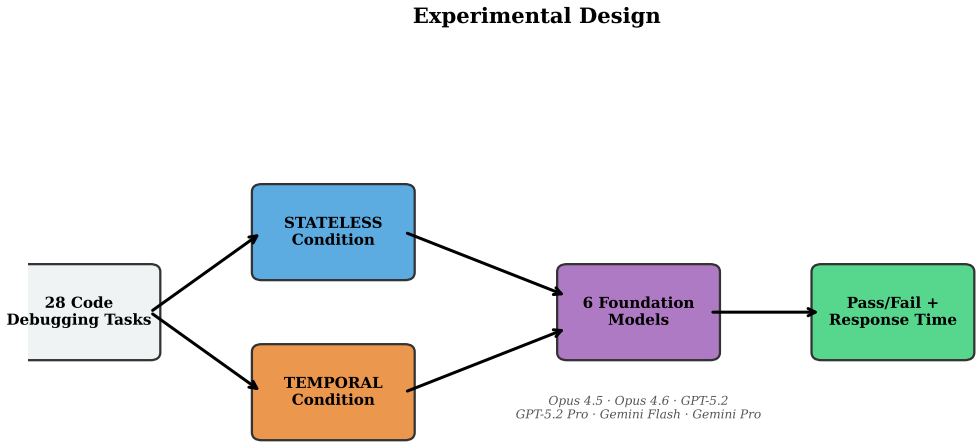

Figure 6: Experimental design overview. 28 debugging tasks are presented in two conditions (STATELESS and TEMPORAL) to six foundation models, measuring pass/fail and response time.